

Otimização automatizada de código MIPS: Detecção de conflitos e reordenamentos de instruções

Helthe M Lima¹, Rafael da Silva¹, Vinícius Medeiros do Nascimento

¹Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brazil

raphacademic@gmail.com , helthelima3@gmail.com , vini1264@gmail.com

Resumo. *Este artigo apresenta o desenvolvimento de um script em Python para análise e otimização de código MIPS. A ferramenta detecta dependências de dados do topo Leitura-após-Escrita (RAW) que causam bolhas no pipeline e aplica estratégias de otimização, incluindo a inserção de NOPs e o reordenamento de instruções, visando garantir a corretude da execução e maximizar o desempenho em um pipeline de 5 estágios.*

1. Introdução

A arquitetura MIPS utiliza uma organização em pipeline para aumentar o paralelismo de instruções. Um pipeline clássico de 5 estágios (IF, ID, EX, MEM, WB) permite que múltiplas instruções sejam processadas simultaneamente. No entanto, essa técnica está sujeita a conflitos (hazards), sendo o mais comum a dependências de dados RAW (Read-After-Write), onde uma instrução anterior tenha concluído a escrita [Patterson e Hennessy 2014].

Para resolver esse conflito de independência sem hardware de adiantamento(forwarding), é necessário inserir bolhas (NOPs) no pipeline, o que reduz o desempenho. Este trabalho propõe uma abordagem de software para identificar automaticamente essas dependências e reorganizar o código para minimizar o impacto no desempenho.

2. O que é pipeline

Pipeline é uma forma de organizar trabalho em estágios sequenciais, onde a saída de um estágio vira a entrada do próximo — parecido com uma linha de montagem. O pipeline em arquiteturas de processador é uma técnica de paralelismo em nível de instrução que divide a execução de cada instrução em estágios sequenciais (tipicamente: *IF* — Instruction Fetch, *ID* — Instruction Decode, *EX* — Execute, *MEM* — Memory, *WB* — Write Back). Cada estágio processa uma parte diferente de instruções distintas ao mesmo tempo, de modo que, em condições ideais, uma nova instrução entra no pipeline a cada ciclo de clock. Essa organização aumenta o throughput do processador — isto é, a taxa de instruções completadas por unidade de tempo — sem aumentar

significativamente a latência de cada instrução individual. Entretanto, o ganho efetivo depende de quão bem as instruções podem progredir em paralelo pelos estágios do pipeline.

3. Problema das dependências

Dependências entre instruções (por exemplo, RAW — Read After Write, WAR — Write After Read, WAW — Write After Write, e dependências de controle por desvios) comprometem o fluxo contínuo do pipeline. Quando uma instrução precisa do resultado produzido por outra que ainda não completou o estágio adequado, o pipeline deve introduzir atrasos (bolhas, stalls) ou empregar mecanismos de encaminhamento (forwarding) e execução fora de ordem para preservar a correção. Dependências de memória (loads/stores) e instruções de ponto flutuante com latências maiores (como divisões) são fontes frequentes de bolhas. Além disso, instruções de desvio podem invalidar instruções já parcialmente carregadas no pipeline, exigindo técnicas como predição de desvios ou preenchimento de delay slots. No contexto de código MIPS simples e de pipelines típicos de cinco estágios, essas dependências traduzem-se diretamente em aumento do número de ciclos para completar uma sequência de instruções, reduzindo o desempenho observado.

4. Objetivo

O presente trabalho tem como objetivo analisar trechos de código MIPS, identificar automaticamente dependências que geram gargalos no pipeline e aplicar transformações que reduzam bolhas e melhorem o throughput. As transformações incluem detecção de dependências (RAW / WAR / WAW), inserção seletiva de instruções nop quando necessário, reordenação de instruções independentes e desdobramento (loop unrolling) de laços para expor paralelismo instrucional. Espera-se produzir um script em Python que leia códigos MIPS, reporte dependências, gere versões otimizadas e estime a contagem de ciclos antes e depois das otimizações. A avaliação do projeto consiste em comparar o número de ciclos e a estrutura do pipeline (hipotético de 5 estágios) para os trechos analisados, documentando ganhos, limitações e situações em que heurísticas locais não são suficientes, formando assim os artefatos exigidos (código, relatórios e resultados) para o repositório do projeto.

5. Metodologia

Arquitetura do Software

O otimizador foi implementado em Python 3.10, utilizando apenas bibliotecas padrão (`os`, `re`). A solução é modularizada em quatro componentes principais, descritos a seguir.

A. Módulo Parser (`parse_instruction`)

Responsável pela análise léxica e semântica de cada linha de instrução.

- Entrada: Uma *string* contendo a linha de código Assembly (ex: "`ADD $t0, $t1, $t2`").
- Processamento: Utiliza Expressões Regulares (`regex`) para isolar o *Opcode* e os operandos. Implementa uma lógica de distinção de tipos de instrução:
 - *Aritmética (R-Type/I-Type)*: Identifica o primeiro operando como destino (escrita) e os restantes como fonte (leitura).
 - *Memória (Store) e Branch*: Identifica todos os operandos como fontes (leitura), pois estas instruções não escrevem em registradores de uso geral.
 - *Memória (Load)*: Identifica o registrador alvo como destino e o registrador de base (dentro dos parênteses) como fonte.
- Saída: Uma tupla contendo (`Opcode`, `Destino`, `[Lista de Fontes]`).

B. Detector de Dependências (`detect_dependencies`)

Realiza a análise estática para identificar Data Hazards.

- Lógica: Mantém um dicionário `last_write` que mapeia cada registrador ao índice da instrução que o escreveu por último. Ao analisar uma nova instrução, calcula a distância $\Delta = \text{Índice}_{\text{atual}} - \text{Índice}_{\text{produtor}}$.
- Critério: Se $\Delta < 3$ (limiar configurável para pipeline de 5 estágios), uma dependência RAW é registrada.

C. Motor de Otimização (`optimize_pipeline`)

Este é o núcleo do sistema, implementando uma heurística de escalonamento de lista (List Scheduling) com lookahead.

- Gestão de Ciclos: Mantém um mapa `reg_ready_cycle` indicando em qual ciclo absoluto de *clock* um registrador estará disponível.
- Detecção de Bolhas: Para cada instrução candidata, calcula: $\text{Bolhas} = \max(0, \text{Ciclo}_{\text{disponível}} - \text{Ciclo}_{\text{atual}})$.
- Algoritmo de Reordenação: Se $\text{Bolhas} > 0$, o algoritmo busca nas próximas 5 instruções (janela de *lookahead*) uma candidata para

adiantamento. Uma instrução só é movida se passar em três verificações de segurança:

- Sem Conflito Global: Os seus operandos fonte já estão prontos.
- Sem Conflito Local: Não conflita com a instrução que causou a bolha original.
- Sem Violação de Ordem (WAR/WAW): Não conflita com nenhuma instrução intermediária que seria "saltada" (proteção contra *Write-After-Read* e *Write-After-Write*).
- Fallback: Se nenhuma reordenação for segura, instruções **NOP** são inseridas.

- Linguagem usada: Python
- IDE usada (VSCode)
- Leitura de arquivos MIPS
- Algoritmo de detecção
- Estratégia de otimização

4. Resultados e Testes

4.1. Códigos MIPS analisado (A)

```
(A)
ADD $t0, $t1, $t2
SUB $t3, $t0, $t4
AND $t5, $t3, $t6
OR  $t7, $t5, $t8
```

- Dependências encontradas

```

[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por $t0 (produzido na 0). Distância: 1
Linha 1 espera por $t0 (produzido na 0). Distância: 1
Linha 2 espera por $t3 (produzido na 1). Distância: 1
Linha 3 espera por $t5 (produzido na 2). Distância: 1

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_A.txt
-----

Prévia do Código Final:
ADD $t0, $t1, $t2
OR $t7, $t5, $t8      # [REORDENADO]
SUB $t3, $t0, $t4
NOP                   # [BOLHA]
AND $t5, $t3, $t6

```

De acordo com a execução do analisador, foram identificadas as seguintes dependências:

- Linha 1 espera pelo registrador \$t0, produzido na linha 0. Distância: 1
- Linha 2 espera pelo registrador \$t0, produzido na linha 0. Distância: 1
- Linha 2 espera pelo registrador \$t3, produzido na linha 1. Distância: 1
- Linha 3 espera pelo registrador \$t5, produzido na linha 2. Distância: 1

- **Código otimizado**

```

codigo_pipeline > ≡ otimizado_codigo_A.txt
1  ADD $t0, $t1, $t2
2  OR $t7, $t5, $t8      # [REORDENADO]
3  SUB $t3, $t0, $t4
4  NOP                   # [BOLHA]
5  AND $t5, $t3, $t6

```

Análise

O Código A apresenta uma sequência de instruções aritméticas com dependências encadeadas, o que resulta em múltiplos hazards RAW quando executado em um pipeline MIPS clássico.

A aplicação de reordenamento de instruções independentes permitiu ocupar ciclos que seriam desperdiçados por stalls, reduzindo significativamente o número total de ciclos de execução.

Mesmo não sendo possível eliminar completamente todas as dependências, a inserção controlada de uma bolha garantiu a correção semântica do programa, ao mesmo tempo em que melhorou o desempenho global do pipeline.

Este teste valida a eficácia do otimizador em cenários simples de dependência de dados e estabelece uma base sólida para testes mais complexos envolvendo instruções de memória e controle de fluxo.

4.2. Código MIPS analisado (B)

```
(B)
ADD R1, R2, R3
SUB R4, R1, R5
AND R6, R1, R7
OR R8, R1, R9
XOR R10, R1, R11
```

- Dependências encontradas

```
[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por R1 (produzido na 0). Distância: 1
Linha 2 espera por R1 (produzido na 0). Distância: 2
```

- Código otimizado

```
codigo_pipeline > ≡ otimizado_codigo_B.txt.txt
1  ADD R1, R2, R3
2  NOP          # [BOLHA]
3  SUB R4, R1, R5
4  AND R6, R1, R7
5  OR R8, R1, R9
6  XOR R10, R1, R11
```

Análise

O Código B apresenta um padrão clássico de dependência, onde várias instruções consomem o valor de um mesmo registrador produzido por uma única instrução inicial.

Nesse cenário, o reordenamento não é eficaz, pois todas as instruções dependem diretamente de R1, impossibilitando a movimentação segura de instruções independentes para preencher os ciclos de espera.

Dessa forma, a única estratégia viável aplicada pelo otimizador foi a inserção de uma bolha (NOP) logo após a instrução produtora, garantindo que o valor de R1 estivesse disponível para as instruções consumidoras subsequentes.

O teste evidencia uma limitação natural do escalonamento estático quando não há instruções independentes suficientes para exploração do paralelismo de instruções (ILP).

```
[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por R1 (produzido na 0). Distância: 1
Linha 2 espera por R1 (produzido na 0). Distância: 2

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_B.txt.txt
-----

Prévia do Código Final:
ADD R1, R2, R3
NOP          # [BOLHA]
SUB R4, R1, R5
AND R6, R1, R7
OR  R8, R1, R9
XOR R10, R1, R11
```

O teste B (Figura 2) simulou um cenário onde um valor (R1) é calculado e logo em seguida utilizado por várias instruções (SUB, AND, OR).

- Observação: O otimizador inseriu apenas um NOP logo após a instrução produtora (ADD).
- Análise de Pipeline: Esta solução demonstra eficiência. Ao inserir uma bolha entre o produtor e o primeiro consumidor (SUB), o script atrasa o fluxo o suficiente para resolver o conflito crítico. As instruções subsequentes (AND, OR), por estarem naturalmente sequenciadas após o SUB, já encontram o dado R1 disponível no banco de registradores quando chegam ao seu estágio ID, não necessitando de bolhas adicionais.

4.4. Código MIPS analisado (C)

O teste C (Figura 3) envolveu uma instrução de carga (**L.D**), que possui alta latência, dentro de um laço de repetição.

- **Observação:** O script identificou que a instrução de aritmética inteira **DADDUI**, originalmente no final do bloco, era independente. Ela foi movida para logo após o **L.D** (marcada como **[REORDENADO]**).
- **Análise de Pipeline:** Instruções de *Load* buscam o dado no estágio **MEM** (4º estágio). Se a instrução seguinte tentar usar esse dado no estágio **EX**, ocorrerá um bloqueio. Ao mover o **DADDUI** para preencher esse intervalo, o processador executa trabalho útil (atualização de ponteiro/contador) durante o ciclo que, de outra forma, seria desperdiçado esperando pela memória. Isso maximiza a utilização dos estágios do pipeline.

```
[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por F0 (produzido na 0). Distância: 1

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_D.txt.txt
-----

Prévia do Código Final:
DIV.D F0, F2, F4
SUB.D F12, F8, F14      # [REORDENADO]
ADD.D F10, F0, F8
```

(imagem D.png como Figura 3)

4.5. Código MIPS analisado (D)

O teste D (Figura 4) apresentou um cenário de operações de ponto flutuante, onde uma divisão (**DIV.D**) bloqueava uma soma dependente (**ADD.D**).

- **Observação:** O script localizou uma subtração independente (**SUB.D**) que estava mais à frente no código e moveu-a para entre as instruções conflitantes.
- **Análise de Pipeline:** Este é o cenário ideal de *Instruction Scheduling*. Ao reordenar o código, a ferramenta eliminou completamente a necessidade de **NOPs**. Enquanto a unidade de ponto flutuante processa a **DIV.D** (que pode levar múltiplos ciclos no estágio EX), o pipeline não para; ele continua a despachar e executar a instrução **SUB.D**. O

resultado é um CPI (Ciclos Por Instrução) mais próximo de 1, mantendo todos os estágios do pipeline ativos.

```
PS C:\Dev\Analise pipeline 2> & C:\Python313\python.exe "c:/Dev/Analise pipeline 2/otimizador_mips.py"
--- Processando: codigo_D.txt.txt ---

[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por F0 (produzido na 0). Distância: 1

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_D.txt.txt
-----

Prévia do Código Final:
DIV.D F0, F2, F4
SUB.D F12, F8, F14      # [REORDENADO]
ADD.D F10, F0, F8
```

(imagem D.png como Figura 4)

4.6. Código MIPS analisado (E)

O quinto teste (Código E) apresentou um desafio complexo envolvendo operações de ponto flutuante com interdependências de escrita e leitura cruzadas.

- Código de Entrada (Reconstruído):

1. DIV.D F0, F2, F4 (Escreve em F0)
2. ADD.D F6, F0, F8 (Lê F0 e F8; Escreve em F6)
3. S.D F6, 0(R1) (Lê F6)
4. SUB.D F8, F10, F14 (Escreve em F8)
5. MUL.D F6, F10, F8 (Lê F8)

- Análise da Execução (Figura 5):

A ferramenta identificou duas situações críticas de pipeline:

1. Dependência RAW (F0): O ADD.D precisa do resultado da DIV.D. Como não havia instruções seguras para mover, o script inseriu um NOP.
2. Oportunidade de Reordenação (F6): O S.D precisa do resultado do ADD.D. Originalmente, haveria uma bolha aqui. O script

identificou que a instrução SUB.D (originalmente na linha 4) era independente do par ADD/S.D e moveu-a para cima ([REORDENADO]).

```
[DEPENDÊNCIAS ENCONTRADAS]
Linha 1 espera por F0 (produzido na 0). Distância: 1
Linha 2 espera por F6 (produzido na 1). Distância: 1
Linha 4 espera por F8 (produzido na 3). Distância: 1

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_E.txt
-----

Prévia do Código Final:
DIV.D F0, F2, F4
NOP                # [BOLHA]
ADD.D F6, F0, F8
SUB.D F8, F10, F14    # [REORDENADO]
S.D F6, 0(R1)
MUL.D F6, F10, F8
```

- **Destaque - Proteção WAR (Write-After-Read):**

Um ponto crucial deste teste foi o que o script NÃO fez.

Poder-se-ia pensar em mover o SUB.D para preencher o primeiro buraco (logo após o DIV.D). No entanto, o ADD.D lê o registrador F8, e o SUB.D escreve no F8.

Se o script movesse o SUB.D para antes do ADD.D, ocorreria uma violação WAR (Write-After-Read): o ADD.D leria o novo valor de F8 (produzido pela subtração) em vez do valor original. O algoritmo detectou este conflito e corretamente impediu a reordenação insegura, optando pelo NOP na primeira lacuna e reordenando apenas na segunda, onde era seguro.

- **Resultado:**

A execução final combinou segurança e eficiência:

- DIV $\$to\$$ NOP $\$to\$$
- ADD $\$to\$$
- SUB (Preenche latência) $\$to\$$ S.D.

4.7. Código MIPS analisado (F)

O teste E (Figura 5) envolveu múltiplas dependências cruzadas entre operações de ponto flutuante.

- Observação: O script aplicou uma estratégia híbrida: inseriu um **NOP** após o **DIV.D**, mas conseguiu reordenar o **SUB.D** para depois do **ADD.D**.
- Análise de Pipeline: O algoritmo demonstrou robustez ao evitar reordenações inseguras. Mover o **SUB.D** para muito cedo poderia causar um conflito *Write-After-Read* (WAR) em registradores compartilhados. A ferramenta optou por garantir a integridade dos dados (via **NOP**) na primeira dependência crítica, mas aproveitou a oportunidade para reordenar na segunda parte do bloco, equilibrando segurança e desempenho.

```
[DEPENDÊNCIAS ENCONTRADAS]
Nenhuma dependência crítica.

[GERANDO ARQUIVO OTIMIZADO]
-----
SUCESSO! Código otimizado salvo em:
C:\Dev\Analise pipeline 2\codigo_pipeline\otimizado_codigo_F.txt
-----

Prévia do Código Final:
ADD R4, R5, R6
BEQ R1, R2, EXIT
OR R7, R8, R9
```

Figura 5. Execução do cenário E: Estratégia mista de NOP e Reordenação segura.

O teste F (Figura 6) submeteu à análise um trecho de código contendo uma instrução aritmética, seguida de um desvio condicional (*branch*) e uma operação lógica, sem compartilhamento de registradores entre elas.

- Observação: O script analisou as instruções **ADD**, **BEQ** e **OR**. A saída indicou "Nenhuma dependência crítica". Consequentemente, o código otimizado gerado é idêntico ao original, sem inserção de **NOPs** ou reordenações desnecessárias.
- Análise de Pipeline: Este cenário ilustra o funcionamento ideal do pipeline de 5 estágios.

1. A instrução **ADD** escreve em **R4** no estágio WB.
2. A instrução seguinte, **BEQ**, lê **R1** e **R2** no estágio ID.
3. Como os registradores de leitura (**R1**, **R2**) são distintos do registrador de escrita da instrução anterior (**R4**), não há conflito de dados (Hazard RAW). O hardware pode despachar uma instrução por ciclo (CPI = 1) sem paradas. O fato de o script não ter alterado o código valida a sua precisão: ele intervém apenas quando estritamente necessário para garantir a integridade dos dados, preservando a eficiência original quando possível.

5. Conclusão

O desenvolvimento deste otimizador de código MIPS permitiu a aplicação prática dos conceitos fundamentais de Arquitetura de Computadores, especificamente no que tange ao funcionamento de pipelines de 5 estágios e o tratamento de conflitos (*hazards*).

Foi possível constatar, através dos experimentos realizados, que a análise estática de código é uma ferramenta poderosa para garantir a integridade da execução em processadores que não dispõem de mecanismos complexos de *interlock* ou *forwarding* via hardware. Os testes comparativos demonstraram que, embora a inserção de NOPs (realizada nos Cenários A e B) resolva efetivamente os conflitos de dados RAW, ela impõe uma penalidade no tamanho do código e no tempo de execução. Em contrapartida, a técnica de reordenação de instruções (*Instruction Scheduling*), aplicada com sucesso nos Cenários C e D, provou ser superior ao ocultar as latências de acesso à memória e operações aritméticas, maximizando o *throughput* do pipeline sem alterar o número de ciclos de clock necessários.

Limitações

Apesar dos resultados satisfatórios, o algoritmo implementado apresenta limitações inerentes à sua abordagem heurística. A estratégia de busca gulosa (*Lookahead*) restrita a uma janela de instruções futuras é eficaz para otimizações locais (dentro de um bloco básico), mas não garante o ótimo global que seria obtido através da construção de um Grafo de Dependência de Dados (DAG) completo. Além disso, o modelo atual assume latências fixas simplificadas para operações de ponto flutuante e não trata dependências de controle complexas, limitando-se a garantir a segurança em desvios simples.

6. References

Patterson, D. A. e Hennessy, J. L. (2014) Computer Organization and Design: The Hardware/Software Interface. 5th Edition. Morgan Kaufmann.

[UFRR] Material de aula da disciplina Arquitetura e Organização de Computadores. Departamento de Ciência da Computação, UFRR.